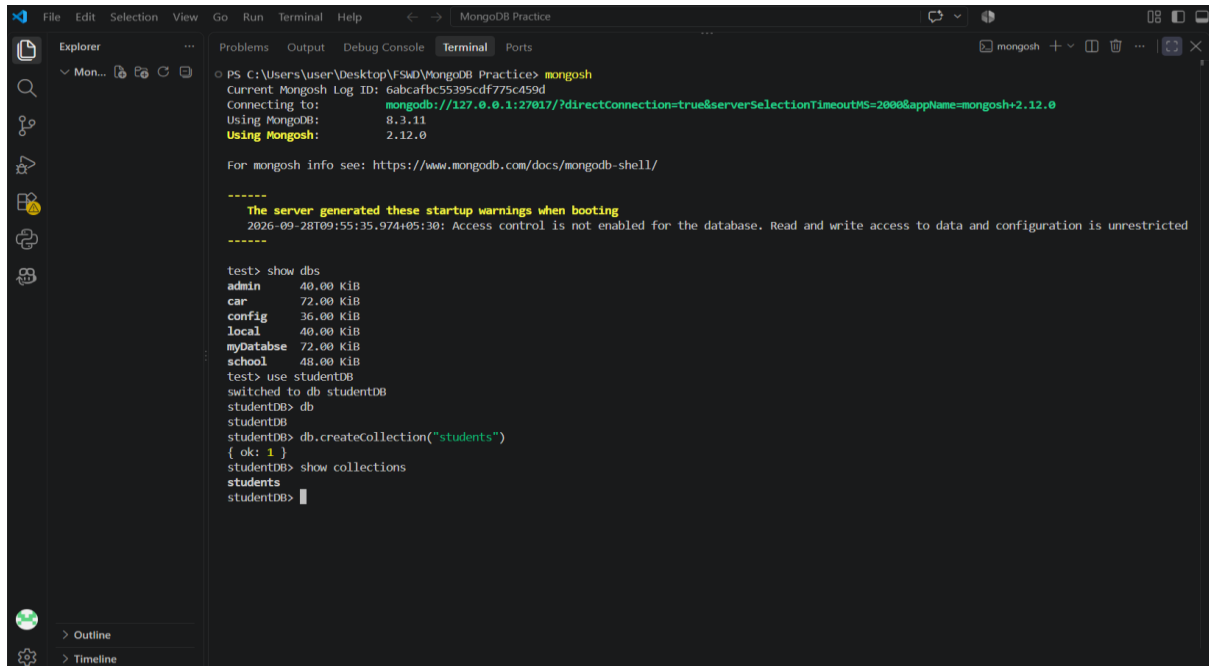


MongoDB Practice

1. Database and Collection Setup

show dbs listed the existing databases. use studentDB switched to the new database studentDB. db confirmed the current database. db.createCollection("students") created the students collection, and show collections displayed students.

A screenshot of a VS Code terminal window titled 'MongoDB Practice'. The terminal shows the execution of 'mongosh' and various MongoDB commands. It lists existing databases, switches to 'studentDB', creates a 'students' collection, and lists the collections. A warning message about access control is also visible.

```
PS C:\Users\User\Desktop\FSD\MongoDB Practice> mongosh
Current Mongosh Log ID: 6abcafb55395cdf775c459d
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB: 8.3.11
Using Mongosh: 2.12.0

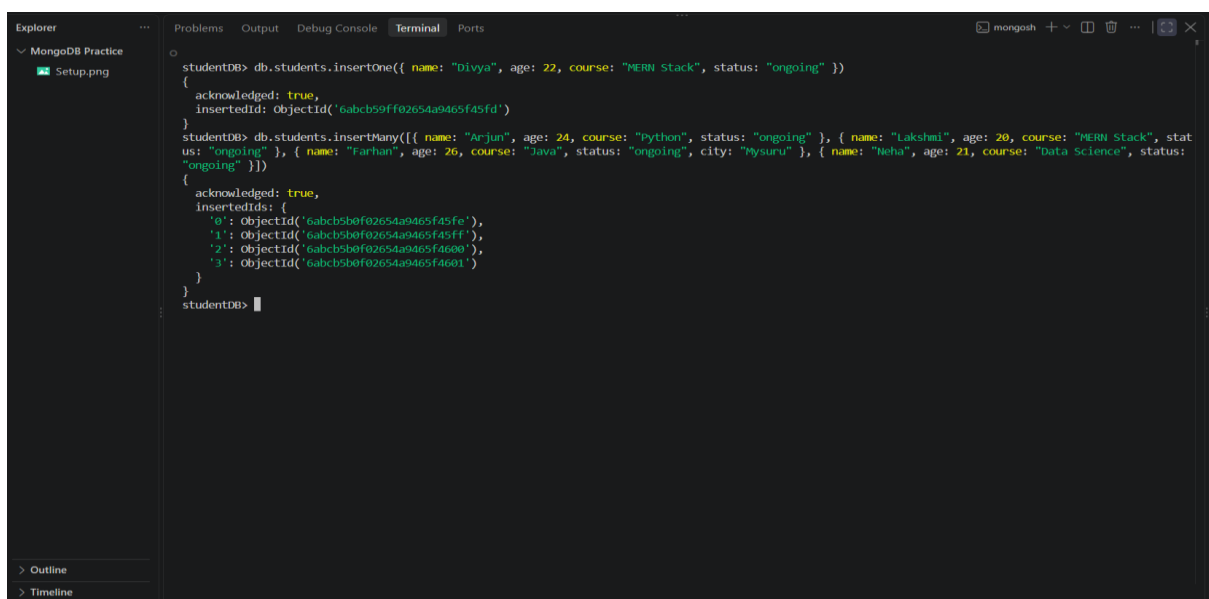
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-09-28T09:55:35.974+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> show dbs
admin      40.00 KiB
car        72.00 KiB
config     36.00 KiB
local      40.00 KiB
myDatabase 72.00 KiB
school     48.00 KiB
test> use studentDB
switched to db studentDB
studentDB> db
studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show collections
students
studentDB>
```

2. Insert Operations

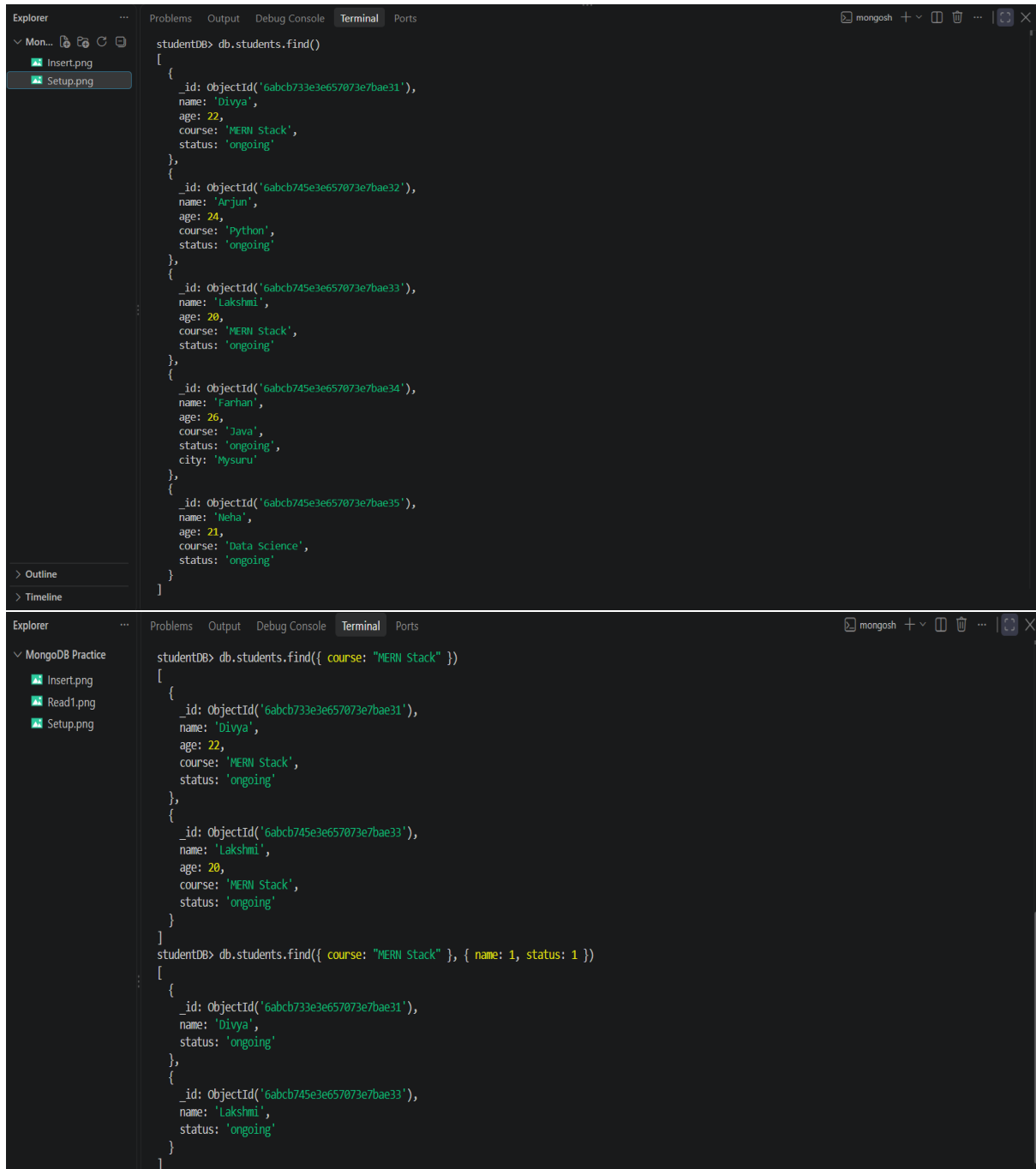
insertOne added one student, Divya (age 22, MERN Stack, ongoing). insertMany added four more: Arjun (Python), Lakshmi (MERN Stack), Farhan (Java, city Mysuru), and Neha (Data Science).

A screenshot of a VS Code terminal window titled 'MongoDB Practice'. The terminal shows the execution of 'insertOne' and 'insertMany' commands to add students to the 'students' collection. The output shows the acknowledgment of the insert operations and the inserted IDs.

```
studentDB> db.students.insertOne({ name: "Divya", age: 22, course: "MERN Stack", status: "ongoing" })
{ acknowledged: true, insertedId: ObjectId('6abcb59ff02654a9465f45fd') }
studentDB> db.students.insertMany([
  { name: "Arjun", age: 24, course: "Python", status: "ongoing" },
  { name: "Lakshmi", age: 20, course: "MERN Stack", status: "ongoing" },
  { name: "Farhan", age: 26, course: "Java", status: "ongoing", city: "Mysuru" },
  { name: "Neha", age: 21, course: "Data Science", status: "ongoing" }
])
{ acknowledged: true, insertedIds: [
  ObjectId('6abcb5b0f02654a9465f45fe'),
  ObjectId('6abcb5b0f02654a9465f45ff'),
  ObjectId('6abcb5b0f02654a9465f4600'),
  ObjectId('6abcb5b0f02654a9465f4601')
] }
studentDB>
```

3. Read Operations

find() fetched all 5 students. The filter { course: "MERN Stack" } returned only Divya and Lakshmi. The select object { name: 1, status: 1 } showed only their name and status. { age: 0 } showed all students without the age field. findOne returned Neha, and countDocuments returned 2 MERN Stack students.



The image shows two screenshots of a VS Code editor with a MongoDB terminal window. The top screenshot shows the command `studentDB> db.students.find()` and its output, which is a JSON array of five student documents. The bottom screenshot shows two more commands: `studentDB> db.students.find({ course: "MERN Stack" })` and `studentDB> db.students.find({ course: "MERN Stack" }, { name: 1, status: 1 })`, along with their respective outputs.

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae32'),
    name: 'Arjun',
    age: 24,
    course: 'Python',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae35'),
    name: 'Neha',
    age: 21,
    course: 'Data Science',
    status: 'ongoing'
  }
]
```

```
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

```
studentDB> db.students.find({ course: "MERN Stack" }, { name: 1, status: 1 })
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    status: 'ongoing'
  }
]
```

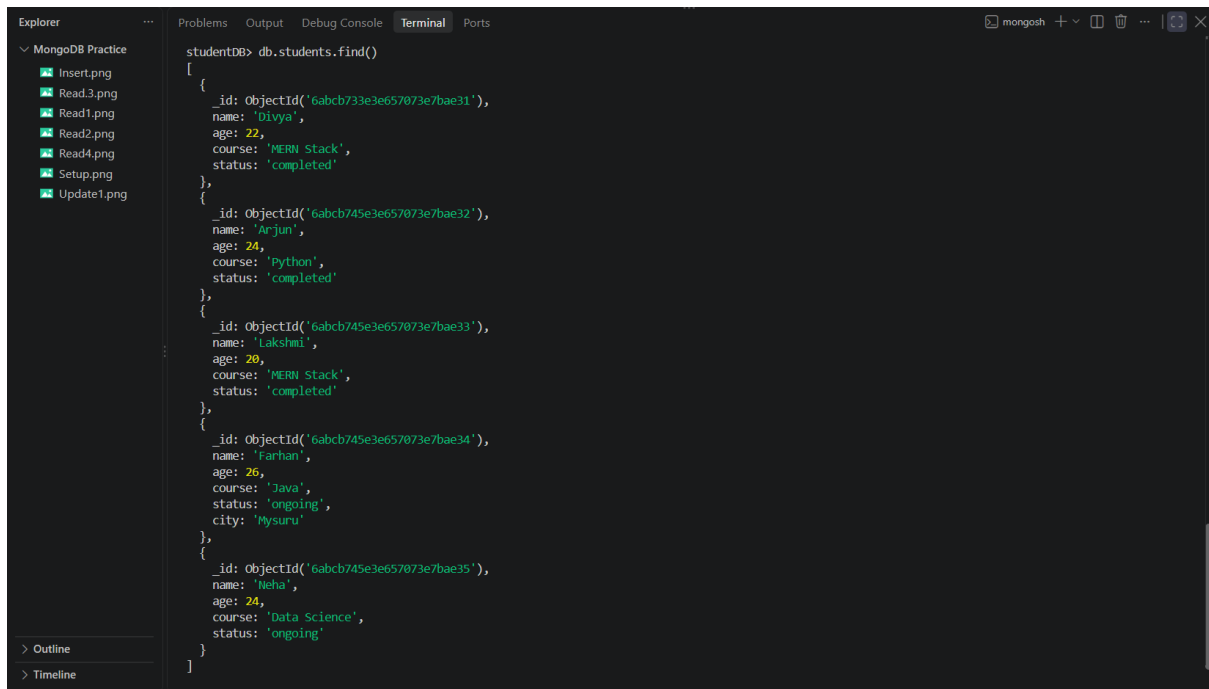
```
studentDB> db.students.find({}, { age: 0 })
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae32'),
    name: 'Arjun',
    course: 'Python',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae35'),
    name: 'Neha',
    course: 'Data Science',
    status: 'ongoing'
  }
]
```

```
studentDB> db.students.findOne({ name: "Neha" })
{
  _id: ObjectId('6abcb745e3e657073e7bae35'),
  name: 'Neha',
  age: 21,
  course: 'Data Science',
  status: 'ongoing'
}
studentDB> db.students.countDocuments({ course: "MERN Stack" })
2
studentDB> 
```

4. Update Operations

updateOne changed Arjun's status from "ongoing" to "completed". updateMany changed the status of both MERN Stack students, Divya and Lakshmi, to "completed". \$inc added 3 to the age of the Data Science student, so Neha's age went from 21 to 24. The final find() shows the updated data.

```
studentDB> db.students.updateOne({ name: "Arjun" }, { $set: { status: "completed" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany({ course: "MERN Stack" }, { $set: { status: "completed" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
studentDB> db.students.updateMany({ course: "Data Science" }, { $inc: { age: 3 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```



The screenshot shows a MongoDB terminal window with the following command and output:

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae32'),
    name: 'Arjun',
    age: 24,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae35'),
    name: 'Neha',
    age: 24,
    course: 'Data Science',
    status: 'ongoing'
  }
]
```

5. Query Operators

\$gt: 22 returned Arjun (24), Farhan (26), and Neha (24), the students older than 22.

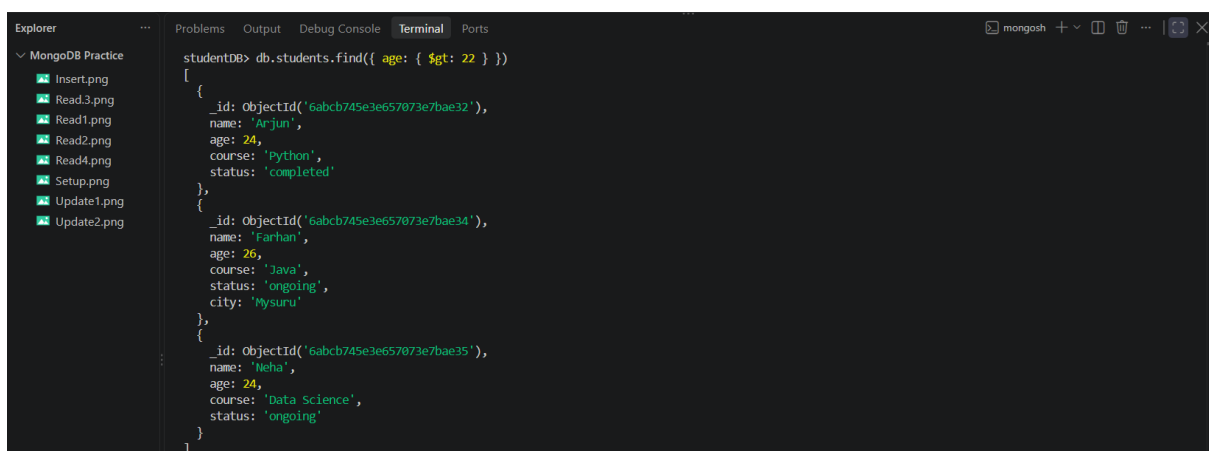
\$lt: 23 returned Divya (22) and Lakshmi (20), the students younger than 23.

\$in: ["Python", "Java"] returned Arjun (Python) and Farhan (Java).

\$and (MERN Stack and completed) returned Divya and Lakshmi.

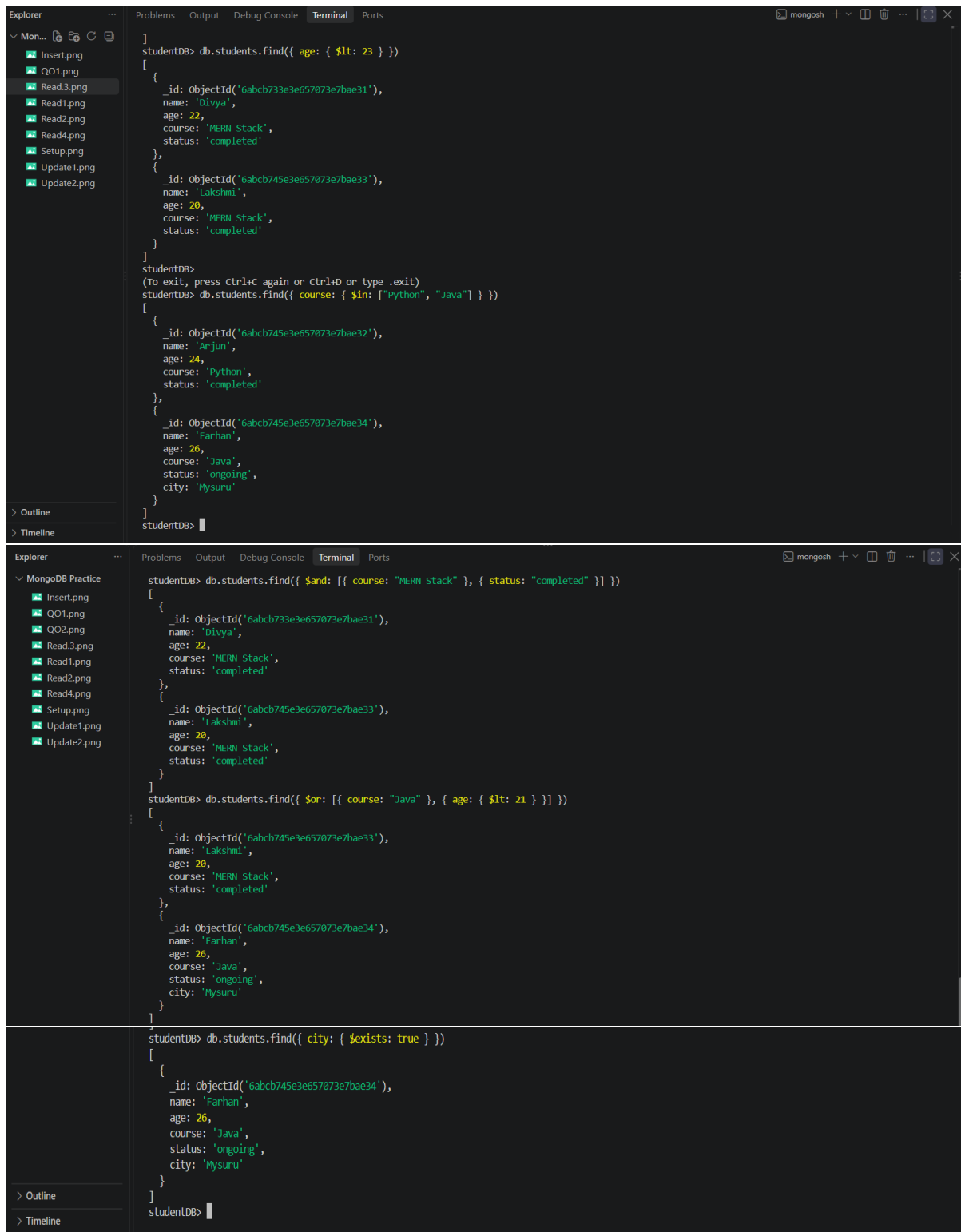
\$or (Java, or age below 21) returned Farhan (Java) and Lakshmi (age 20).

\$exists: true on city returned only Farhan, the one.



The screenshot shows a MongoDB terminal window with the following command and output:

```
studentDB> db.students.find({ age: { $gt: 22 } })
[
  {
    _id: ObjectId('6abcb745e3e657073e7bae32'),
    name: 'Arjun',
    age: 24,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae35'),
    name: 'Neha',
    age: 24,
    course: 'Data Science',
    status: 'ongoing'
  }
]
```



The image displays three sequential screenshots of a MongoDB terminal window, showing various find queries and their results. The terminal interface includes a sidebar with file explorer and a main area for the terminal output.

First Screenshot: The terminal shows a find query for students with an age of 23. The results are two documents: one for Divya (age 22, completed) and one for Lakshmi (age 20, completed).

```
studentDB> db.students.find({ age: { $lt: 23 } })
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
studentDB> db.students.find({ course: { $in: ["Python", "Java"] } })
[
  {
    _id: ObjectId('6abcb745e3e657073e7bae32'),
    name: 'Arjun',
    age: 24,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  }
]
studentDB>
```

Second Screenshot: The terminal shows a find query for students with a course of 'MERN Stack' and a status of 'completed'. The results are two documents: one for Divya (age 22, completed) and one for Lakshmi (age 20, completed).

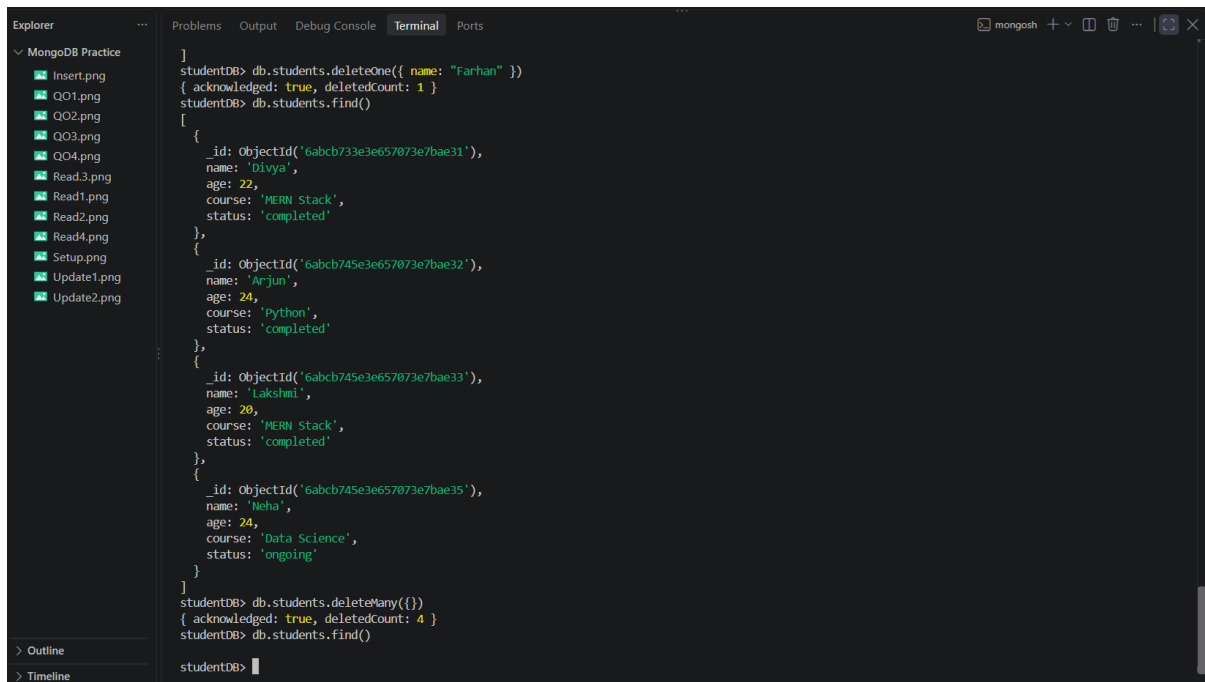
```
studentDB> db.students.find({ $and: [{ course: "MERN Stack" }, { status: "completed" }] })
[
  {
    _id: ObjectId('6abcb733e3e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB> db.students.find({ $or: [{ course: "Java" }, { age: { $lt: 21 } }] })
[
  {
    _id: ObjectId('6abcb745e3e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  }
]
studentDB>
```

Third Screenshot: The terminal shows a find query for students with a city that exists. The results are one document: one for Farhan (age 26, ongoing, Mysuru).

```
studentDB> db.students.find({ city: { $exists: true } })
[
  {
    _id: ObjectId('6abcb745e3e657073e7bae34'),
    name: 'Farhan',
    age: 26,
    course: 'Java',
    status: 'ongoing',
    city: 'Mysuru'
  }
]
studentDB>
```

6. Delete Operations

`deleteOne({ name: "Farhan" })` removed Farhan, leaving Divya, Arjun, Lakshmi, and Neha. `deleteMany({})` then removed all remaining students, and the last `find()` returned `[]`, so the collection is empty.

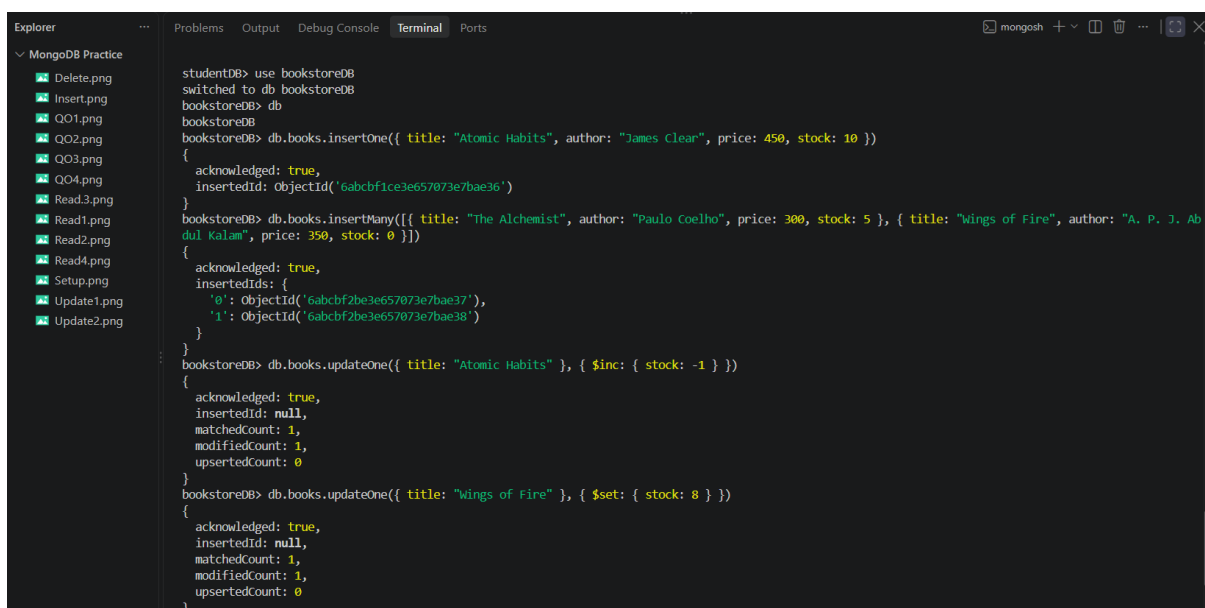


```
studentDB> db.students.deleteOne({ name: "Farhan" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abcb733e657073e7bae31'),
    name: 'Divya',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e657073e7bae32'),
    name: 'Arjun',
    age: 24,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e657073e7bae33'),
    name: 'Lakshmi',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abcb745e657073e7bae35'),
    name: 'Neha',
    age: 24,
    course: 'Data Science',
    status: 'ongoing'
  }
]
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB> db.students.find()
studentDB>
```

7. Use Case: Online Bookstore

The bookstoreDB database holds a books collection. insertOne added "Atomic Habits" (price 450, stock 10). insertMany added "The Alchemist" (300, stock 5) and "Wings of Fire" (350, stock 0). When a copy of Atomic Habits was sold, \$inc: { stock: -1 } reduced its stock from 10 to 9. When Wings of Fire was restocked, \$set changed its stock from 0 to 8.

Searches: stock > 0 returned all three books, author: "Paulo Coelho" returned The Alchemist, and the \$or query (price below 320, or title "Wings of Fire") returned The Alchemist and Wings of Fire.



```
studentDB> use bookstoreDB
switched to db bookstoreDB
bookstoreDB> db
bookstoreDB
bookstoreDB> db.books.insertOne({ title: "Atomic Habits", author: "James Clear", price: 450, stock: 10 })
{ acknowledged: true,
  insertedId: ObjectId('6abcbf2be3e657073e7bae36') }
bookstoreDB> db.books.insertMany([ { title: "The Alchemist", author: "Paulo Coelho", price: 300, stock: 5 }, { title: "Wings of Fire", author: "A. P. J. Abdul Kalam", price: 350, stock: 0 } ])
{ acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abcbf2be3e657073e7bae37'),
    '1': ObjectId('6abcbf2be3e657073e7bae38')
  } }
bookstoreDB> db.books.updateOne({ title: "Atomic Habits" }, { $inc: { stock: -1 } })
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
bookstoreDB> db.books.updateOne({ title: "Wings of Fire" }, { $set: { stock: 8 } })
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
```

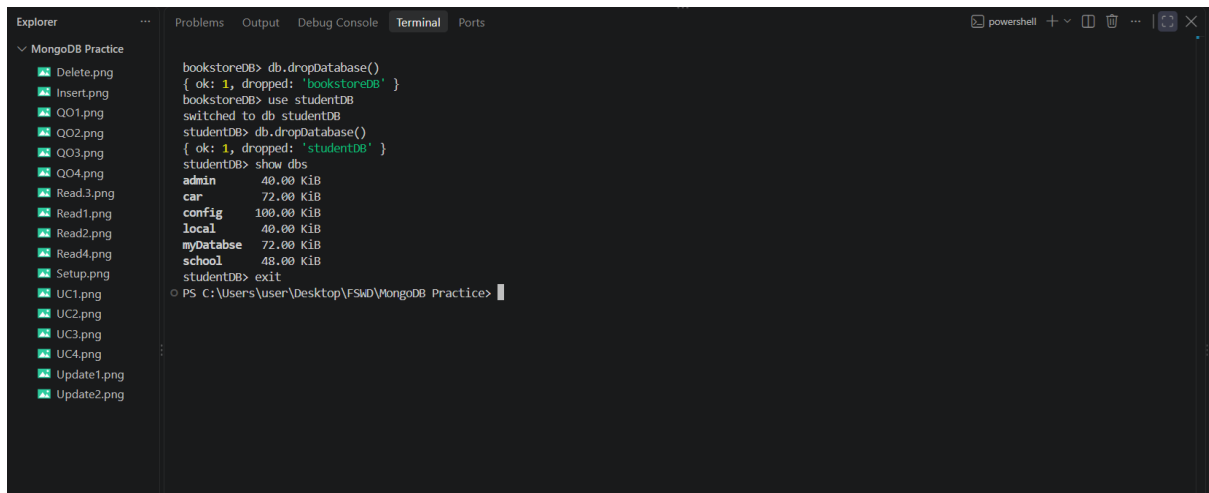
```
bookstoreDB> db.books.find({ stock: { $gt: 0 } })
[
  {
    _id: ObjectId('6abcbf1ce3e657073e7bae36'),
    title: 'Atomic Habits',
    author: 'James Clear',
    price: 450,
    stock: 9
  },
  {
    _id: ObjectId('6abcbf2be3e657073e7bae37'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6abcbf2be3e657073e7bae38'),
    title: 'Wings of Fire',
    author: 'A. P. J. Abdul Kalam',
    price: 350,
    stock: 8
  }
]
bookstoreDB> db.books.find({ author: "Paulo Coelho" })
[
  {
    _id: ObjectId('6abcbf2be3e657073e7bae37'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    price: 300,
    stock: 5
  }
]
```

```
bookstoreDB> db.books.find({ $or: [{ price: { $lt: 320 } }, { title: "Wings of Fire" }] })
[
  {
    _id: ObjectId('6abcbf2be3e657073e7bae37'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6abcbf2be3e657073e7bae38'),
    title: 'Wings of Fire',
    author: 'A. P. J. Abdul Kalam',
    price: 350,
    stock: 8
  }
]
bookstoreDB>
```

```
bookstoreDB> db.books.find({ $or: [{ price: { $lt: 320 } }, { title: "Wings of Fire" }] })
[
  {
    _id: ObjectId('6abcbf2be3e657073e7bae37'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6abcbf2be3e657073e7bae38'),
    title: 'Wings of Fire',
    author: 'A. P. J. Abdul Kalam',
    price: 350,
    stock: 8
  }
]
bookstoreDB>
```

8. Cleanup

`db.dropDatabase()` deleted the practice databases, show dbs confirmed they were gone, and exit closed mongosh.



The screenshot shows a VS Code interface with a terminal window open. The Explorer sidebar on the left shows a folder named 'MongoDB Practice' containing various image files. The terminal window, titled 'powershell', shows the following commands and output:

```
bookstoreDB> db.dropDatabase()
{ ok: 1, dropped: 'bookstoreDB' }
bookstoreDB> use studentDB
switched to db studentDB
studentDB> db.dropDatabase()
{ ok: 1, dropped: 'studentDB' }
studentDB> show dbs
admin          40.00 KiB
car            72.00 KiB
config         100.00 KiB
local          40.00 KiB
myDatabase     72.00 KiB
school         48.00 KiB
studentDB> exit
PS C:\Users\user\Desktop\FSD\MongoDB Practice>
```